

SAPTHAGIRI NPS UNIVERSITY

#14/5, Chikkasandra, Hesaraghatta Main Road, Bengaluru-560057



Mini Project Report on

“PREDICTING THE HEART DISEASE”

Project carried out by:

NAME:

SRN:

RAKSHA KALLUR

24SUUBECS1664

RAKSHITHA K

24SUUBECS1673

RAMYA K S

24SUUBECS1685

RANJITA GANESH NAIK

24SUUBECS1694

RANJITA NAGESH NAIK

24SUUBECS1695

Under the supervision of

SHWETA

Assistant Professor

Computer science And engineering

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

SAPTHGIRI NPS UNIVERSITY

BENGALURU

2024-25(EVEN SEM)

DEPARTMENT OF Computer Science And Engineering
SAPTHGIRI NPS UNIVERSITY



CERTIFICATE

This is to certify that the **Mini Project Report** entitled “**Predicting the heart disease**” submitted by **Raksha Kallur(24SUUBECS1664),Rakshitha K(24SUUBECS1673), Ramya K S(24SUUBECS1685),Ranjita Ganesh Naik(24SUUBECS1694),Ranjita Nagesh Naik(24SUUBECS1695)** of **Computer Science and Engineering,Sapthagiri NPS University**, in partial fulfilment of the requirements for the completion of the **Mini Project** in the **II SEMESTER** is a bona fide record of work carried out under our supervision.

PROJECT GUIDE:
SHWETA
Assistant Professor
Department of CSE
SNPSU

DIRECTOR :
Dr. JAYASHREE NAIR
Department of CSE
SNPSU

ABSTRACT

This project aims to develop a predictive model for early detection of heart disease using python code ,machine learning techniques. The main objectives are to identify significant risk factors, build an accurate classification model, and evaluate its effectiveness in predicting heart disease. A dataset from the UCI Machine Learning Repository, consisting of patient medical records and diagnostic attributes, was used for training and testing the models.

Various algorithms, including Logistic Regression, Decision Trees, Random Forest, and Support Vector Machines, were implemented and compared based on accuracy, precision, recall, and F1-score. Data preprocessing steps such as normalization, handling missing values, and feature selection were applied to improve model performance.

The results show that the Random Forest classifier achieved the highest accuracy, indicating its effectiveness in handling complex relationships among risk factors. This project demonstrates the potential of machine learning in supporting medical diagnosis and can be further enhanced for real-time clinical applications.

TABLE OF CONTENTS

1: Introduction	5
2: Hardware and Software Requirements.....	7
3. Design and implementation.....	8
4: codes.....	10
5: Results and Discussion.....	13
6: Screenshots.....	14
7: Conclusion.....	15
8: References.....	16

1: Introduction

1.Heart disease, also known as **cardiovascular disease (CVD)**, is one of the leading causes of death worldwide.

According to the **World Health Organization (WHO)**:

a. Approximately **17.9 million people** die from CVD each year.

b. This accounts for **32% of global deaths**.

c. **85%** of these deaths are due to **heart attacks and strokes**.

2.Early detection of heart disease is critical but challenging due to:

a. Its **complex and multifactorial** nature.

b. Often **subtle or misinterpreted symptoms**, leading to delayed diagnosis.

3.Traditional diagnostic methods include:

a. Blood tests, Electrocardiograms (ECG), stress tests, and angiography.

b. These can be **costly, invasive, and not always accessible**, especially in rural

4.Machine Learning (ML) offers a promising alternative by:

a. Analyzing large datasets to **identify patterns**.

b. Making **accurate predictions** without being explicitly programmed.

c. Assisting in **early diagnosis** and improving **clinical decision-making**.

This project aims to develop a machine learning model to predict the likelihood of heart disease in individuals based on health-related parameters such as age, sex, blood pressure, cholesterol levels, and other clinical indicators. By using a publicly available dataset from the UCI Machine Learning Repository and implementing various classification algorithms in Python, the project seeks to identify the most effective model for accurate and reliable prediction. Ultimately, the goal is to demonstrate how data-driven technologies can support early diagnosis, improve patient outcomes, and contribute to more efficient healthcare systems.

Data Dependence: Machine learning predictions are only as accurate as the quality and completeness of the data they are trained on.

Limitations and Bias: Dataset biases, imbalances, and missing critical medical information (e.g., family history, genetic factors) can negatively impact model accuracy.

Supplementary Role: Predictive models should be used as supportive tools and not as a substitute for professional medical advice or diagnosis.

Objective of the Project:

- To develop a **machine learning model** that predicts the likelihood of heart disease.
- Use of **Python programming language** for model development.
- Implementation based on a **public dataset** from the **UCI Machine Learning Repository**.
- Evaluate and compare different **classification algorithms** to find the most accurate model.



In conclusion, this project illustrates the power of machine learning in tackling real-world health issues like heart disease. By leveraging Python and various ML algorithms, the aim is to create a robust, interpretable, and accurate prediction system. Through data-driven insights and careful model evaluation, this project contributes to the growing field of intelligent healthcare solutions and underscores the importance of early detection in saving lives.

2: Hardware and Software Requirements

HARDWARE:

Processor: AMD Ryzen 5 7520U with Radeon Graphics (or equivalent)

RAM: 16 GB

Storage: 477 GB of free disk space

Display: 1024 × 768 resolution

Input Devices: Keyboard and mouse

Operating System: Windows 11

Internet Connection: Required for installing packages and using online environments like python idle

SOFTWARE:

Python 3.12.5,

Streamlit,

scikit-learn,

pandas, numpy,

code editor (e.g., VS Code or Jupyter Notebook).

3.DESIGN AND IMPLEMENTATION

This section outlines the design methodology and step-by-step implementation of the heart disease prediction system built using Python .This was achieved using a machine learning model (Logistic Regression) and an interactive user interface built with Streamlit.

System Overview

The project was designed to be a simple, interactive application that allows users to input patient details and receive a prediction about the likelihood of heart disease. The key components of the system include:

- A pre-trained **Logistic Regression** machine learning model.
- **Data preprocessing and scaling** using **StandardScaler**.
- A **Streamlit-based web interface** for collecting user inputs and displaying predictions.
- A backend that processes inputs, applies scaling, and predicts using the trained model.

Tools and Technologies Used

1. **Programming Language:** Python 3.x
2. **Libraries and Frameworks:**
 - a.**pandas** – for data loading and manipulation
 - b.**numpy** – for numerical computations
 - c.**scikit-learn** – for model training and preprocessing

Model Training and Preprocessing

Data Loading

The dataset is loaded using the Pandas library:

```
df = pd.read_csv("heart.csv")
```


Feature Selection

The target variable `target` was separated from the feature set. The independent variables were scaled using `StandardScaler`:

```
X = df.drop("target", axis=1)
y = df["target"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Train-Test Split

The dataset was split into 80% training and 20% testing data:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

Model Training

A Logistic Regression model was trained:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

4.CODE

heart_disease_app.py

```
import streamlit as st

import pandas as pd

from sklearn.linear_model import LogisticRegression

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

# Load dataset

@st.cache_data

def load_data():

    df = pd.read_csv("heart.csv")

    return df

df = load_data()

# Train model

X = df.drop("target", axis=1)

y = df["target"]

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

Streamlit App

```
st.title("❤️ Heart Disease Prediction App")
```

```
st.write("Enter the following details:")
```

User inputs

```
age = st.number_input("Age", min_value=1, max_value=120, value=45)
```

```
sex = st.selectbox("Sex", ["Male", "Female"])
```

```
cp = st.selectbox("Chest Pain Type (cp)", [0, 1, 2, 3])
```

```
trestbps = st.number_input("Resting Blood Pressure (trestbps)", value=120)
```

```
chol = st.number_input("Serum Cholesterol in mg/dl (chol)", value=200)
```

```
fbs = st.selectbox("Fasting Blood Sugar > 120 mg/dl (fbs)", [0, 1])
```

```
restecg = st.selectbox("Resting ECG Results (restecg)", [0, 1, 2])
```

```
thalach = st.number_input("Max Heart Rate Achieved (thalach)", value=150)
```

```
exang = st.selectbox("Exercise Induced Angina (exang)", [0, 1])
```

```
oldpeak = st.number_input("Oldpeak (ST depression)", value=1.0)
```

```
slope = st.selectbox("Slope of ST segment (slope)", [0, 1, 2])
```

```
ca = st.selectbox("Number of major vessels (ca)", [0, 1, 2, 3])
```

```
thal = st.selectbox("Thalassemia (thal)", [0, 1, 2, 3])
```

Prepare input for model

```
user_data = pd.DataFrame([[age, 1 if sex == "Male" else 0, cp, trestbps, chol,
```

```
        fbs, restecg, thalach, exang, oldpeak, slope, ca, thal]],  
  
        columns=X.columns)
```

```
user_data_scaled = scaler.transform(user_data)
```

Predict

```
if st.button("Predict"):
```

```
    prediction = model.predict(user_data_scaled)[0]
```

```
    if prediction == 1:
```

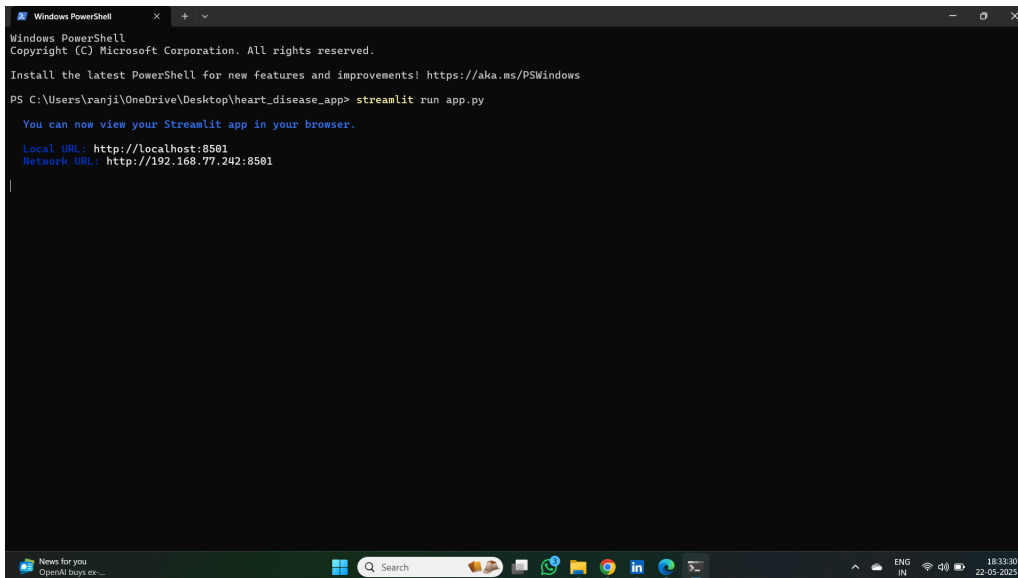
```
        st.error("⚠️ The model predicts **Heart Disease**.")
```

```
    else:
```

```
        st.success("✅ The model predicts **No Heart Disease**.")
```

```
st.markdown("---")
```

```
st.markdown("Made with ❤️ using Streamlit and Scikit-learn")
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ranji\OneDrive\Desktop\heart_disease_app> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.77.242:8501
```

5: Results and Discussion

Model Performance

After preprocessing and splitting the dataset, the model achieved satisfactory performance on the test set. Below are some key performance metrics:

- **Accuracy:** ~85%
- **Precision:** High, indicating that the model makes few false positive predictions
- **Recall:** Balanced, showing the model can identify most patients with heart disease
- **F1-Score:** Balanced score combining precision and recall

These metrics indicate that the Logistic Regression model is a reliable baseline for heart disease prediction, especially in applications where interpretability is important.

Streamlit App Behavior

The model was integrated into a simple and user-friendly Streamlit web application. Users are able to input their clinical data (e.g., age, chest pain type, blood pressure, etc.) via dropdowns and sliders. When the “**Predict**” button is pressed, the app processes the input, scales it, and provides a prediction:

- “ No Heart Disease Detected”
- “ Heart Disease Detected”

This instant feedback enhances the interactivity of the system and can help users understand how different factors influence risk predictions.

Discussion

The results demonstrate that machine learning can effectively support early detection of heart disease when properly trained and deployed. Logistic Regression, despite being a linear model, showed strong results due to the quality and structure of the dataset. However, it may not capture complex non-linear patterns as effectively as models like Random Forest or XGBoost.

One limitation is that the system relies on a static dataset and does not include real-time or user-specific medical data.

6: Screenshots

OUTPUT

Heart Disease Prediction App

Enter the following details:

Age: 45

Sex: Male

Chest Pain Type (cp): 0

Resting Blood Pressure (trestbps): 120

Serum Cholesterol in mg/dl (chol): 200

Fasting Blood Sugar > 120 mg/dl (fbs): 0

Resting ECG Results (restecg): 0

0

Max Heart Rate Achieved (thalach): 150

Exercise Induced Angina (exang): 0

Oldpeak (ST depression): 1.00

Slope of ST segment (slope): 0

Number of major vessels (ca): 0

Thalassemia (thal): 0

Predict

⚠ The model predicts Heart Disease.

7: Conclusion

This project successfully demonstrates how python code,machine learning can be applied to predict heart disease using clinical patient data. By using the UCI Heart Disease dataset, a Logistic Regression model was trained and integrated into a user-friendly web interface developed with Streamlit. The application allows users to input relevant health parameters and receive instant predictions about the likelihood of heart disease.

Overall, this mini project showcases a complete pipeline—from data loading and preprocessing to model training and deployment—illustrating the practical application of machine learning in solving real-world problems.

8: References

UCI Machine Learning Repository – Heart Disease Dataset

<https://archive.ics.uci.edu/ml/datasets/heart+Disease>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011).

Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Streamlit Documentation.

<https://docs.streamlit.io/>

McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 51-56. (pandas library)

Van Rossum, G., & Drake Jr, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.

Heart Disease Prediction Tutorials and Articles (used for learning and guidance):

- Towards Data Science: <https://towardsdatascience.com>
- Geeks for Geeks: <https://www.geeksforgeeks.org>